MENU-DRIVEN STRING OPERATIONS

Reversing Each Word of a Sentence
and Printing the Initials of a Name

Submitted By

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a menu-driven C program, using separate functions, to perform the following operations:

- Reverse each word in a sentence without changing the word order. Example: Input: What is special?, Output: tahw si ?laiceps
- Print the initials of a name. Example: Input: Alexander Grahambell, Output: A.G.

2. Objective

The objective of this project is to practise string handling in C by writing a modular, menu-driven program. Each task is written as a separate function so that the code is easy to read, test and reuse.

3. Concepts Used

Concept	Where it is used
Functions	Every task has its own function (reversing, initials, menu display, input reading).
Character arrays (strings)	The sentence and the name are stored in a char array of size 200.
do-while loop and switch	The menu keeps repeating until the user chooses Exit.
Library <ctype.h>	isspace(), tolower() and toupper() are used to process characters.
Library <string.h>	strlen() and strcspn() are used to handle the input line.
fgets() and sscanf()	Safe input of a full line that contains spaces.

4. Algorithm

4.1 Main Program (Menu)

Step 1. Start.

Step 2. Display the menu: 1. Reverse each word, 2. Print initials, 3. Exit.

Step 3. Read the choice of the user.

Step 4. If the choice is 1, read a sentence, call reverseEachWord() and display the result.

Step 5. If the choice is 2, read a name and call printInitials() to display the initials.

Step 6. If the choice is 3, display a goodbye message and stop the loop.

Step 7. For any other choice, display an "Invalid choice" message.

Step 8. Repeat from Step 2 until the choice is 3.

Step 9. Stop.

4.2 Reverse Each Word

Step 1. Read the sentence into the string str.

Step 2. Convert every character of str to lower case.

Step 3. Set i = 0. Skip all spaces, then set start = i.

Step 4. Move i forward until a space or the end of the string is reached; this marks one word.

Step 5. Reverse the characters from index start to i - 1 by swapping the first and last characters and moving inwards.

Step 6. Repeat Steps 3 to 5 until the end of the string is reached.

Step 7. Display the modified string.

4.3 Print Initials

Step 1. Read the name into the string name and set inWord = 0.

Step 2. Take each character of the name one by one.

Step 3. If the character is a space, set inWord = 0.

Step 4. Otherwise, if inWord is 0, print the character in capital letter followed by a full stop (.) and set inWord = 1.

Step 5. Repeat Steps 2 to 4 until the end of the string.

5. Functions Used in the Program

Function	Purpose
`main()`	Runs the menu in a loop and calls the other functions.
`displayMenu()`	Prints the menu options on the screen.
`readChoice()`	Reads the menu choice safely and returns it as an integer.
`readLine()`	Reads a full line (including spaces) and removes the newline.
`reverseSegment()`	Reverses the characters of a string between two given indices.
`reverseEachWord()`	Reverses every word of the sentence while keeping the word order.
`printInitials()`	Prints the capital initial of every word of a name.

6. Source Code

File name: string_menu.c

```
/*
 * Program : Menu-Driven String Operations
 * Author  : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 *
 * Operations:
 *   1. Reverse each word in a sentence (word order is unchanged)
 *   2. Print the initials of a name
 */

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 200

/* Function prototypes */
void displayMenu(void);
int  readChoice(void);
void readLine(const char prompt[], char str[], int size);
void reverseSegment(char str[], int start, int end);
void reverseEachWord(char str[]);
void printInitials(const char name[]);

int main(void)
{
    char text[MAX];
    int choice;

    do
    {
        displayMenu();
        choice = readChoice();

        switch (choice)
        {
            case 1:
                readLine("Enter a sentence: ", text, MAX);
                if (strlen(text) == 0)
                {
                    printf("No input given. Please try again.\n");
                }
                else
                {
                    reverseEachWord(text);
                    printf("Output: %s\n", text);
                }
                break;

            case 2:
                readLine("Enter a name: ", text, MAX);
                if (strlen(text) == 0)
                {
                    printf("No input given. Please try again.\n");
                }
                else
                {
                    printf("Output: ");
                    printInitials(text);
                    printf("\n");
                }
                break;

            case 3:
                printf("Exiting the program. Thank you!\n");
                break;
```

```
            default:
                printf("Invalid choice! Please enter 1, 2 or 3.\n");
        }
    } while (choice != 3);

    return 0;
}

/* Prints the menu on the screen */
void displayMenu(void)
{
    printf("\n========== MENU ==========\n");
    printf("1. Reverse each word in a sentence\n");
    printf("2. Print initials of a name\n");
    printf("3. Exit\n");
    printf("==========================\n");
}

/* Reads the user's menu choice safely; returns 0 if input is not a number */
int readChoice(void)
{
    char buffer[MAX];
    int choice = 0;

    printf("Enter your choice: ");
    if (fgets(buffer, sizeof(buffer), stdin) == NULL)
        return 3;                       /* End of input: exit gracefully */
    if (sscanf(buffer, "%d", &choice) != 1)
        return 0;
    return choice;
}

/* Reads a full line (with spaces) and removes the trailing newline */
void readLine(const char prompt[], char str[], int size)
{
    printf("%s", prompt);
    if (fgets(str, size, stdin) == NULL)
    {
        str[0] = '\0';
        return;
    }
    str[strcspn(str, "\n")] = '\0';
}

/* Reverses the characters of str between index start and end (inclusive) */
void reverseSegment(char str[], int start, int end)
{
    char temp;

    while (start < end)
    {
        temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

/* Reverses every word of the sentence in place; word order stays the same.
 * The result is printed in lower case, as shown in the problem statement. */
void reverseEachWord(char str[])
{
    int i = 0, start;

    for (int k = 0; str[k] != '\0'; k++)
        str[k] = (char)tolower((unsigned char)str[k]);

    while (str[i] != '\0')
    {
        while (str[i] != '\0' && isspace((unsigned char)str[i]))
            i++;                                    /* skip spaces */
```

```
        start = i;
        while (str[i] != '\0' && !isspace((unsigned char)str[i]))
            i++;                                    /* find end of word */

        if (i > start)
            reverseSegment(str, start, i - 1);
    }
}

/* Prints the first letter of every word of the name in capitals,
 * each followed by a full stop. Example: Alexander Grahambell -> A.G. */
void printInitials(const char name[])
{
    int inWord = 0;

    for (int i = 0; name[i] != '\0'; i++)
    {
        if (isspace((unsigned char)name[i]))
        {
            inWord = 0;
        }
        else if (!inWord)
        {
            printf("%c.", toupper((unsigned char)name[i]));
            inWord = 1;
        }
    }
}
```

7. Sample Output

The following is the output of the program when it is compiled and run:

```
========== MENU ==========
1. Reverse each word in a sentence
2. Print initials of a name
3. Exit
==========================
Enter your choice: 1
Enter a sentence: What is special?
Output: tahw si ?laiceps

========== MENU ==========
1. Reverse each word in a sentence
2. Print initials of a name
3. Exit
==========================
Enter your choice: 2
Enter a name: Alexander Grahambell
Output: A.G.

========== MENU ==========
1. Reverse each word in a sentence
2. Print initials of a name
3. Exit
==========================
Enter your choice: 3
Exiting the program. Thank you!
```

8. Test Cases

Choice	Input	Output	Result
1	What is special?	tahw si ?laiceps	Pass
1	Hello World	olleh dlrow	Pass
1	I love C programming	i evol c gnimmargorp	Pass
2	Alexander Grahambell	A.G.	Pass
2	Mahatma Gandhi	M.G.	Pass
2	rudra pratap singh	R.P.S.	Pass
7	(invalid choice)	Invalid choice message	Pass
1	(empty line)	No input given message	Pass

9. Conclusion

The program successfully performs both operations through a simple menu. Using separate functions made the code modular and easy to understand. Input is read with fgets(), so sentences containing spaces are handled correctly, and invalid choices or empty input are reported without crashing the program. This project helped in understanding string handling, character functions, loops and function-based program design in C.